

Lex: A Programming Language for Developing Enforceable Legal Formalizations

Jonas Degelo, François Hublet, and Jakob Merane

ETH Zürich, Switzerland
{degeloj,fhublet,jmerane}@ethz.ch

Abstract. Keywords: Legal formalization · Programming languages.

1 Introduction

2 Logical foundations

3 Syntax

3.1 Labels

Labels are used to capture the structure of the law. Their syntax is as follows:

$$\begin{aligned} \text{label_level} &::= \text{law} \mid \text{title} \mid \text{chapter} \mid \text{section} \\ &\quad \mid \text{article} \mid \text{paragraph} \mid \text{point} \mid \text{subpoint} \\ \text{label} &::= \text{label_level}[\text{int}] \text{ string } [\text{string}] \end{aligned}$$

The optional integer parameter that follows the level can be used to encode an arbitrary number of additional hierarchical levels: `chapter[1]` is one level below `chapter` but still above `section`, `chapter[i]` is one level below `chapter[i - 1]` and one level above `chapter[i + 1]`, etc. The second and (optional) third argument collect the number and header of the corresponding part of the law.

Example 1. Art. 2 GDPR can be introduced as

```
article "2" "Material scope"
```

3.2 Types

Lex has four base types: `int` (integers), `float` (floating-point numbers), `bool` (booleans), and `str` (strings). New types can be declared as follows:

$$\text{type_decl} ::= \text{type ident } [[\text{is|subtype}] \text{ ident}]$$

When another, already existing type name is specified after the `is` keyword, the newly declared type is created as a copy of that existing type. As a consequence, the new type will have the same internal representation and take the same values

as the existing type. However, the existing type and the new type will still be considered distinct types by Lex's type system (see example below). In contrast, the `subtype` keyword support specifying a new type as a subtype of an existing type. In this case, the new type will have both the same internal representation and the capacity to be used in any context where the existing type could be used.

Example 2. Consider the following declarations:

```
type id is string
type user_id subtype id
```

Further, consider an event with signature $A(b:id, c:string)$ and a variable `uid` of type `user_id`. The expression $A(uid, "foo")$ is type-correct, since `user_id` is a subtype of `id`. The expression $A(uid, uid)$ is not type-correct, since `user_id` is not a subtype of `string` (because `id` is not a subtype of `string`). On the other hand, $A("foo", "foo")$ is type-correct since the type `id` can contain string values.

3.3 Events

The syntax for event declarations is as follows:

```
capabilities ::= observable | suppressable | causable | internal
               | observable causable | causable observable
               | suppressable causable | causable suppressable
event_decl ::= capabilities event ident
              [ "event_desc" ]
              ident11 : ident12
              ...
              identk1 : identk2
```

where *event_desc* is a string that can contain special placeholders $\{ident_{i1}\}$ for $1 \leq i \leq k$. The capabilities associated to an event are organized into the following lattice, where $a \rightarrow b$ denotes that a is strictly weaker than b .

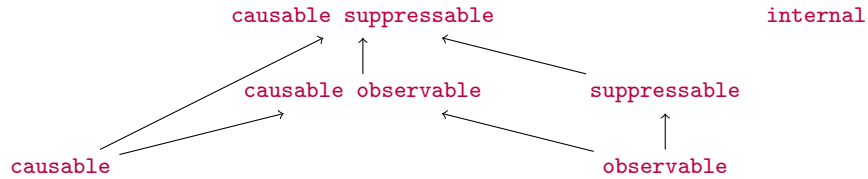


Fig. 1: Capability lattice

The pairs of identifiers $ident_{i1} : ident_{i2}$ contained in the event declaration determine an (ordered) list of event parameters ($ident_{i1}$) and their types ($ident_{i2}$).

Finally, the event description *event_desc* provides the high-level semantics of the event, which should refer to the parameters $ident_{i1}, \dots, ident_{k1}$ using the $\{ident_{i1}\}$ syntax.

Example 3. The following code declares a new suppressable event capturing personal data processing in a system, assuming three previously defined types `processing_id`, `processor_id`, and `data_id`.

```
suppressable event PersonalDataProcessing
"""
processor {x} is processing personal data {z} as part
of processing operation {ep}
"""

    ep : processing_id
    x  : processor_id
    z  : data_id
```

3.4 Rules

The general syntax for rule declarations is as follows:

```
rule_verb ::= oblige | permit | constitute | except
rule_decl ::= rule [string]
              [whenever [pre_opt]
                [label11 :] expr11
                ...
                [label1k :] expr1k]
              rule_verb [post_opt]
                [label21 :] expr21
                ...
                [label2k :] expr2k]
```

In the following we review the syntax of expressions $expr_{ij}$ (Section ??), the available verbs (Section ??) and the temporal patterns determined by *pre_opt* and *post_opt* (Section 3.4).

Temporal patterns. Except for **except** rules, every Lex rule declaration represents a statement of the form

ALWAYS [*pre*($expr_{11}$ **AND** ... **AND** $expr_{1k}$) **IMPLIES** *post*($expr_{21}$ **AND** ... **AND** $expr_{2k}$)]

where *pre* and *post* are determined by *pre_opt* and *post_opt*, respectively. These options use the following grammars for time and time intervals:

time_unit ::= **y** | **d** | **h** | **s** | **ms**
time_bound ::= *int* *time_unit*
time_interval ::= **within** *time_bound* | **before** *time_bound* | **after** *time_bound*

Table 1 lists available (*pre_opt*, *post_opt*) pairs as well as the corresponding *pre* and *post* functions.

Pattern	<i>pre_opt</i>	<i>post_opt</i>	<i>pre</i> (ϕ)	<i>post</i> (ψ)
–	–	–	ϕ	ψ
–	eventually	<i>[time_interval]</i>	ϕ	EVENTUALLY _{<i>I</i>} ψ
–	always	<i>[time_interval]</i>	ϕ	ALWAYS _{<i>I</i>} ψ
TBC....				

Table 1: Available temporal patterns for **Lex** rules

References